Copyright All rights reserved

12. 공용체
(union)

Heesung Oh
https://github.com/3dapi

Heesung Oh
Copyright All rights reserved

- **구조체 (struct) VS 공용체 (union)**

구분	구조체 (struct)	공용체 (union)
메모리	멤버별 독립 공간	하나의 공간 공유
크기	모든 멤버 크기의 합	가장 큰 멤버 크기
접근	모든 멤버 동시에 사용 가능	한 번에 하나만 의미 있음
저장 방식	각각 따로 저장	같은 위치에 덮어쓰기
패딩	발생함	적용됨
용도	데이터 묶음	메모리 절약 / 타입 재해석

Heesung Oh
Copyright All rights reserved

- 구조체 (struct) VS 공용체 (union)

```
union U {
    char c;    // 1바이트
    int  n;    // 4바이트
};
```

```
주소:  0     1     2     3
    [ c ][ ? ][ ? ][ ? ] // 나머지 바이트는 정의 안됨
    [          n        ]
```

Heesung Oh
Copyright All rights reserved

- Union 선언

```
union 공용체_태그
{
    자료형 멤버1;
    자료형 멤버2;
    ...
}; // 반드시 세미콜론을 붙인다.
```

```
union Data {
    int   i;
    float f;
    char  c;
};
```

Heesung Oh
Copyright All rights reserved

- 값 저장
  - 구조체와 동일

```
union Data data;
data.i = 100;   // int로 저장
data.f = 3.14f; // 같은 메모리에 float로 덮어씀 (이전 값 소멸)
data.c = 'A';   // 같은 메모리에 char로 덮어씀 (이전 값 소멸)
```

- 동작
  - 같은 메모리 사용
  - 값 덮어쓰기
  - 마지막 값만 유효

12.1 공용체의 선언과 사용

Heesung Oh
Copyright All rights reserved

- 배열
  - union Data arr[3]
  - 각 요소는 독립적
  - 내부 멤버만 공유

```
arr[0].i = 100;   // 0 번째 요소를 int로 설정
arr[1].f = 1.23f; // 1 번째 요소를 float로 설정
arr[2].c = 'Z';   // 2 번째 요소를 char로 설정
```

- 포인터
  - 구조체와 동일 접근
  - p->i / p->c

```
union Data value;
union Data* p = &value;

p->i = 42;  // 포인터로 접근해 int로 저장
printf("%d\n", p->i);

p->c = 'B'; // 같은 메모리를 char로 저장
printf("%c\n", p->c);
```

12.2 typedef 를 사용한 공용체 선언

Heesung Oh
Copyright All rights reserved

- typedef
  - union 키워드 생략
  - 일반 자료형처럼 사용

- 기존 공용체 + typedef

```
union Data {
  int   i;
  float f;
  char  c;
};
```

```
// typedef 별칭 정의
typedef union Data Data;
typedef union Data* PtrData;

//사용 예:
Data d;  // union Data d; 와 동일
PtrData p = &d;
```

12.2 typedef 를 사용한 공용체 선언

Heesung Oh
Copyright All rights reserved

- 태그 없이 정의, 별칭 부여

```
typedef union {
    int   i;
    float f;
    char  c;
} Data, *PtrData;

// 사용 예:
Data d;          // union { ... } d; 와 동일
PtrData p = &d; // 공용체 포인터
```

12.2 typedef 를 사용한 공용체 선언

Heesung Oh
Copyright All rights reserved

- 태그 + typedef 동일

```
typedef union Data {
    int   i;
    float f;
    char  c;
} Data, *PtrData;

// 사용 예:
Data d; // union Data d; 대신 Data를 직접 사용
```

12.3 공용체 초기화 규칙

Heesung Oh
Copyright All rights reserved

- 기본 규칙
  - → 하나의 멤버만 초기화 가능
- 기본 초기화
  - union Data d = { 10 }
  - 첫 번째 멤버(i) 초기화
- 잘못된 초기화
  - { 10, 3.14f } → 허용되지 않음
- 지정 초기자 (C99)
  - union Data d = { .f = 3.14f }
  - 특정 멤버 초기화 가능
- 주의
  - 여러 멤버 지정 → 허용되지 않음

12.4 공용체의 활용 – Type Punning

Heesung Oh
Copyright All rights reserved

- Type Punning
  - 동일 메모리, 다른 타입 해석
  - 비트 패턴 유지
  - 디버깅 / 전송 활용

```
union Number {
    int   i;
    float f;
};
```

- 주의
  - 컴파일러 의존
  - 이식성 문제 가능
  - 안전한 방법: memcpy()

```
union Number n;
n.f = 3.14f;
printf("%x\n", n.i);  // float 비트 패턴 → 정수 형식 출력
```

12.4.2 공용체 활용 – Tagged Union

Heesung Oh
Copyright All rights reserved

- enum + union
- 타입 + 값 함께 전달
- 안전한 다형성 구현

```c
enum ParamType {
    PARAM_INT,
    PARAM_FLOAT,
    PARAM_PTR
};
union ParamValue {
    int   i;
    float f;
    void* ptr;
};
struct Param {
    enum ParamType type;
    union ParamValue v;
};
```

```c
void ProcessParam(struct Param p)
{
  switch (p.type)
  {
    case PARAM_INT:
      printf("정수 처리: %d\n", p.v.i);
      break;

    case PARAM_FLOAT:
      printf("실수 처리: %.2f\n", p.v.f);
      break;

    case PARAM_PTR:
      printf("포인터 처리: %p\n", p.v.ptr);
      break;
  }
}
```

12.4.3 공용체 활용- 구조체 + union

Heesung Oh
Copyright All rights reserved

- 하나의 데이터를 여러 방식 접근
- struct / 배열 / 정수

```
typedef union {
    struct {
        unsigned char r, g, b, a;
    };
    unsigned char c[4];   // 4바이트 배열로 접근
    unsigned int value;   // 32비트 정수로 접근
} Color;

Color c;
c.r = 255;
c.g = 128;
c.b = 0;
c.a = 255;

printf("%08X\n", c.value);  // 0xFF8000FF
```

12.4.4 공용체 활용 – 비트 필드 결합

Heesung Oh
Copyright All rights reserved

- 비트 필드 결합
  - 비트 단위 접근
  - 전체 값 접근 동시에 가능
- 주의
  - 엔디안 영향 있음
  - 비트 필드 = 컴파일러 의존

```
struct FlagsBits fb = { 0 };

fb.flagA = 1;
fb.flagB = 0;
fb.flagC = 1;

// 개별 비트에 이름으로 접근 가능
printf("A=%u B=%u C=%u\n",
  fb.flagA, fb.flagB, fb.flagC);
```

```
struct FlagsBits {
 unsigned int flagA    : 1;  // 1비트
 unsigned int flagB    : 1;  // 1비트
 unsigned int flagC    : 1;  // 1비트
 unsigned int reserved: 29; // 나머지 비트
};
```

학습 정리

Heesung Oh
Copyright All rights reserved

- 공용체 (union)
  - 하나의 메모리 공유
  - 마지막 값만 유효 → 덮어쓰기 구조
- 구조체 vs 공용체
  - struct : 메모리 분리
  - union : 메모리 공유
- 선언 및 사용
  - struct와 동일 문법
  - . / -> 연산자 사용
  - 배열 / 포인터 사용 가능
- typedef
  - union 키워드 생략
  - 일반 자료형처럼 사용
- 초기화
  - 하나의 멤버만 가능
  - 기본 : 첫 번째 멤버
  - C99 : 지정 초기자
- Type Punning
  - 동일 메모리 다른 타입 해석
  - 비트 패턴 유지
  - 디버깅 / 전송 활용
  - 컴파일러 의존 (이식성 주의)
- Tagged Union
  - enum + union
  - 타입 + 값 함께 전달
  - 안전한 다형성 패턴
- 구조체 / 배열 결합
  - 다양한 방식 접근
  - struct / 배열 / 정수
  - 엔디안 영향
- 비트 필드 결합
  - 비트 단위 접근
  - 전체 값 접근 가능
  - 컴파일러 의존

Enum, 구조체 공용체 실습 과제

Heesung Oh
Copyright All rights reserved

- 과제 1. 태그드 공용체 기반 이벤트 처리
- 과제 2. RGBA 색상 공용체 처리
- 과제 3. 비트 필드 + 공용체 플래그 관리
- 과제 4. 숫자 문자열 자동 파싱 (VariantNumber)